Koneru Lakshmaiah Education Foundation
(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)
Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.
Phone No: 7815926816, www.klh.edu.in

Department of CSE

B.Tech. CSE Trimester -5

Roll Number:2520030609
Name:V.Siri Chandana

Generative AI and AI Platforms

25CS2207E

WEEK 4 – TOKENIZER LAB

Title:Comparison of BPE, WordPiece and SentencePiece Tokenizers

1. Aim

To implement and compare three subword tokenization approaches–Byte Pair Encoding (BPE), WordPiece and SentencePiece–using English, Telugu and Hindi text, and observe how each tokenizer splits text into tokens and how many tokens are produced.

2. Software Requirements

- Python 3.10 or later
- PyCharm IDE
- Active Python virtual environment (.venv)
- Internet connection to download pretrained tokenizer files
- Required Python libraries:
    - transformers
    - sentencepiece

3. Problem Statement

To understand and compare how different tokenization techniques process multilingual text by converting sentences into tokens and numerical token IDs.

4. Theory

Tokenization is the process of converting text into smaller units called tokens. A token can be a complete word, part of a word, punctuation mark or character-related unit. These tokens are converted into numerical IDs that language models can process.

The three tokenizers used are:

- **BPE (Byte Pair Encoding):** GPT-2 uses byte-level BPE, which creates subword units by repeatedly merging frequent adjacent symbol pairs.
- **WordPiece:** Multilingual BERT uses a learned subword vocabulary and splits words into smaller pieces when necessary.
- **SentencePiece:** XLM-R uses SentencePiece with a BPE model. It learns subword units directly from raw text.

5. Procedure

1. Open PyCharm and create a new Python project named GEN AI WEEK-4.
2. Create a Python virtual environment (.venv).
3. Create a requirements.txt file.
4. Add the required libraries transformers and sentencepiece.
5. Install the libraries using the command:
   python -m pip install -r requirements.txt
6. Create a Python file named tokenizer_comparison.py.
7. Import AutoTokenizer from the Transformers library.
8. Define the pretrained tokenizer models:
   - GPT-2 for BPE
   - Multilingual BERT for WordPiece
   - XLM-R for SentencePiece
9. Provide sample sentences in English, Telugu and Hindi.
10. Load the pretrained tokenizers using AutoTokenizer.from_pretrained().
11. Tokenize each sentence and obtain the token IDs.
12. Display the tokens, token IDs and token counts.
13. Run the program using:
    python tokenizer_comparison.py
14. Observe and compare the output for all three languages.

6. Observation

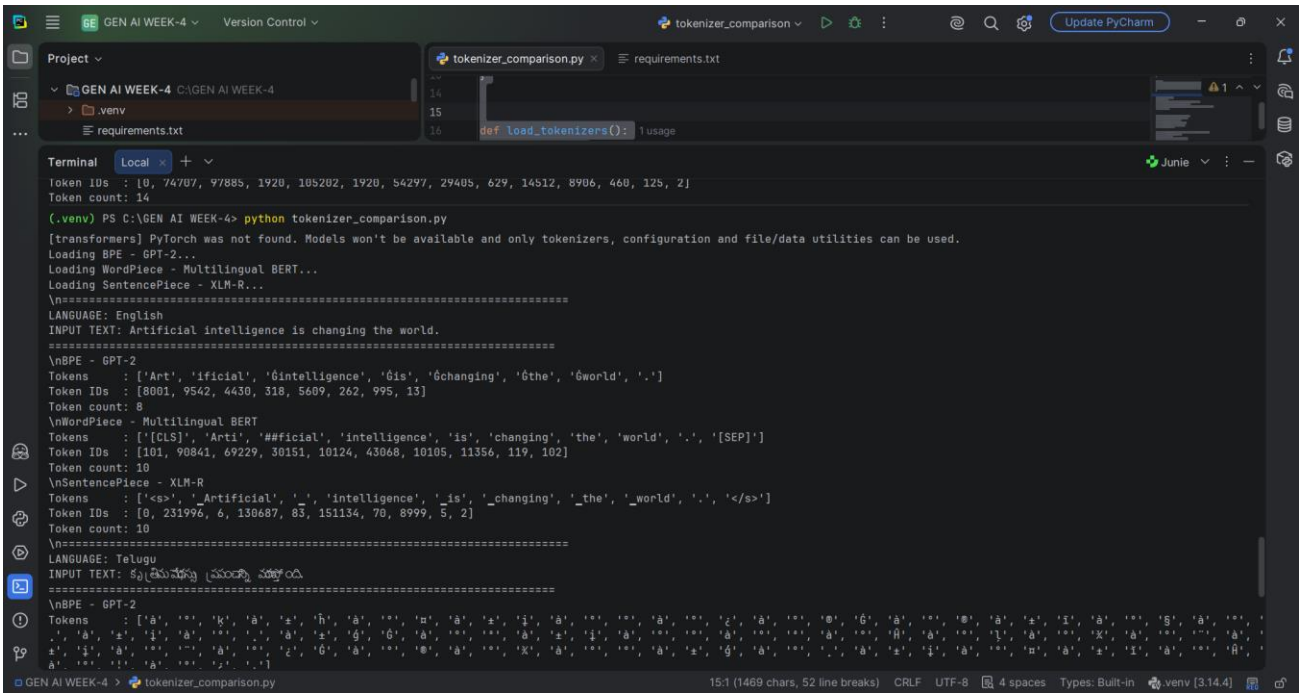
The program compares three pretrained tokenizers using the same English, Telugu and Hindi sentences.

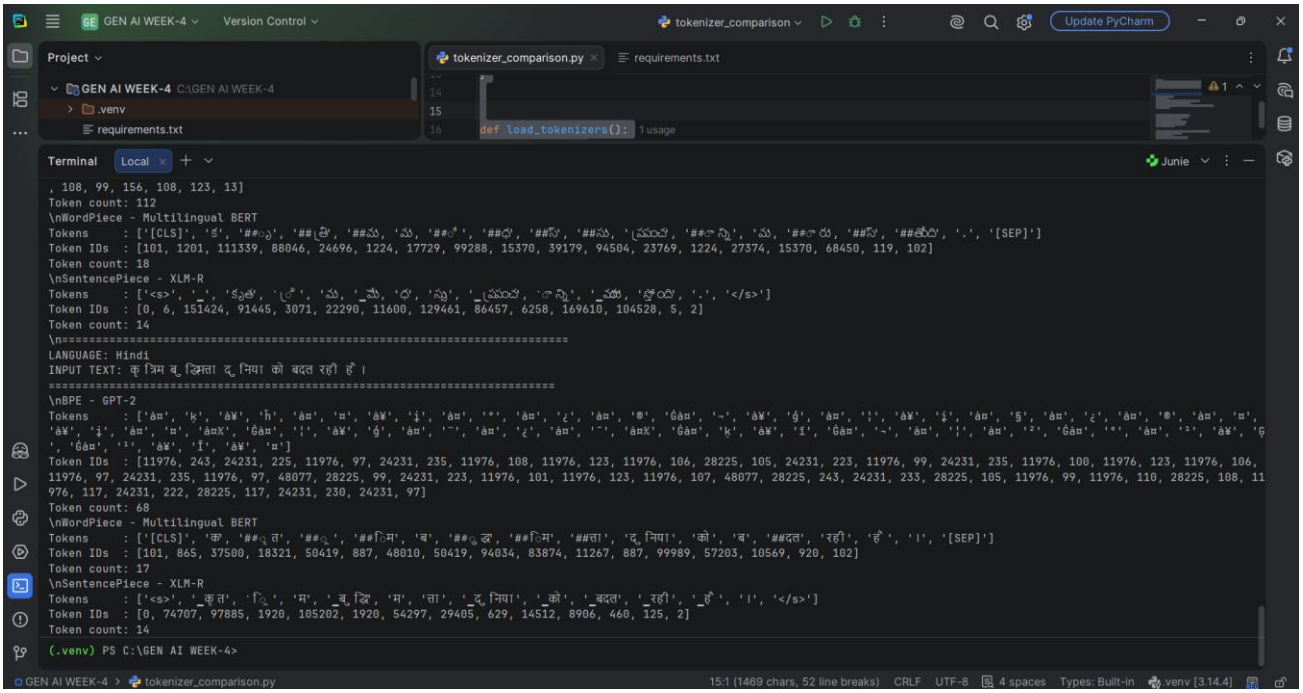
The following details are observed:

- Each tokenizer divides the input text into smaller token units.
- Different tokenizers produce different token pieces and token IDs.
- Token counts vary depending on the tokenizer vocabulary and training data.
- GPT-2 may split Telugu and Hindi text into more tokens than multilingual tokenizers because it is primarily English-trained.

- Multilingual BERT and XLM-R are designed to handle multiple languages.

Output:





Output observation table:

Language	BPE – GPT-2	WordPiece – Multilingual BERT	SentencePiece – XLM-R
English	Record actual token count	Record actual token count	Record actual token count
Telugu	Record actual token count	Record actual token count	Record actual token count
Hindi	Record actual token count	Record actual token count	Record actual token count

Note: Fill in the actual token counts from your program output.

7. Result

Successfully implemented and compared BPE, WordPiece and SentencePiece tokenizers using English, Telugu and Hindi text. Observed the differences in token splitting, token IDs and token counts across the three pretrained tokenizers.

8. Conclusion

This experiment helped us understand the importance of tokenization in Natural Language Processing and how different tokenization approaches handle multilingual text. Token counts depend on the tokenizer vocabulary, algorithm and training data.